Enhanced Autonomous Navigation Using Reinforcement Learning

Vindya Kiran Jain
Department of Artificial Intelligence
REVA College, REVA University
Bengaluru, Karnataka, India
E-mail: Vindya.ai10@race.reva.edu.in

Abstract

This work presents an enhanced experimental study of autonomous navigation in a grid-based environment using tabular Q-learning and a Deep Q-Network (DQN). The implementation evaluates the baseline fixed-obstacle navigation task and extends the study through learning-rate sensitivity, exploration-rate sensitivity, larger-grid scalability, and a dynamic-obstacle experiment. The reproduced 10×10 benchmark uses a start state of (1,1), a goal state of (10,10), four movement actions, fixed obstacles, a goal reward of +100, an obstacle penalty of −100, and a movement cost of −1. Across independent runs, Q-learning achieved 98.99 ± 0.14% success with a mean successful path length of 26.41 ± 20.92 steps, while the reproduced DQN achieved 49.72 ± 17.61% success with 72.79 ± 49.66 steps. The greedy demonstration produced an 18-step route for both learned policies, equal to the Manhattan lower bound for the selected start and goal. Sensitivity experiments showed that increasing α improved Q-learning performance in the tested range, reaching 99.47% success at α=0.50. Increasing grid size caused only a modest decrease in mean success rate but substantially increased path length and delayed convergence. In the dynamic-obstacle experiment, mean success fell to 62.20%, and no run met the convergence criterion within the training horizon. These results indicate that tabular Q-learning is highly effective for the small fixed environment, whereas scalability and environmental non-stationarity remain important challenges.

Keywords

Reinforcement learning, Q-learning, Deep Q-Network, autonomous navigation, grid world, hyperparameter sensitivity, scalability, dynamic obstacles, path planning.

I. Introduction

Autonomous navigation requires an agent to select actions that move it toward a desired destination while avoiding undesirable states. Grid-based environments provide a controlled setting in which navigation policies can be evaluated quantitatively before moving to more complex continuous or physical environments. Classical shortest-path algorithms such as Dijkstra's algorithm and A* provide deterministic solutions when the environment model is known [2], [3], whereas reinforcement learning (RL) learns action values or policies through interaction.

Q-learning is a model-free temporal-difference method that estimates the expected long-term utility of state-action pairs [4]. DQN extends value-based learning by approximating the Q-function with a neural network and using experience replay and a target network to improve training stability [5]. The uploaded implementation reproduces a fixed-obstacle benchmark [1] and then investigates several extensions intended to assess robustness beyond a single small-grid experiment.

The objective of this paper is not to replace the original benchmark results with unsupported claims, but to document the enhanced implementation and its observed results. In particular, the DQN results obtained in the implementation differ from the values reported in the reference paper; the discrepancy is retained and discussed rather than artificially adjusted.

II. Related Work

Shortest-path planning has a long history in artificial intelligence. Dijkstra's algorithm establishes a general shortest-path solution for graphs with non-negative edge costs [2], while A* introduces a heuristic to focus search toward the goal [3]. These methods assume an explicit model of the environment. Reinforcement learning instead allows an agent to learn action preferences from rewards and transitions.

Watkins and Dayan introduced Q-learning as a model-free off-policy temporal-difference algorithm [4]. Sutton and Barto provide the standard theoretical treatment of reinforcement learning [6]. Deep reinforcement learning later enabled neural networks to approximate value functions in high-dimensional state spaces; Mnih et al. demonstrated the DQN framework using experience replay and a target network [5].

III. Objectives

- Reproduce the fixed-obstacle grid-navigation benchmark using the documented Python implementation.
- Compare tabular Q-learning and DQN using success rate, successful path length, and convergence episode.
- Evaluate the effect of Q-learning learning rate α and exploration rate ε.
- Assess Q-learning behavior as grid size increases from 10×10 to 20×20.
- Evaluate navigation under a dynamic-obstacle condition using an expanded Markov state.
- Visualize the learning curves, policy paths, sensitivity results, and scalability trends.
- Identify limitations and practical directions for future autonomous-navigation research.

IV. Problem Statement

The navigation problem is formulated as a finite Markov decision process on an N×N grid [6]. At each time step the agent occupies a cell and chooses one of four actions: right, left, down, or up. The

objective is to reach the goal while minimizing unnecessary movement and avoiding obstacle cells.

For the principal benchmark, the start state is (1,1) and the goal is (10,10). The fixed obstacle coordinates are (3,3), (3,4), (3,5), (4,5), (5,5), (5,6), (6,6), (7,4), (7,5), and (8,5). The reward is +100 on reaching the goal, −100 for an obstacle collision, and −1 for ordinary movement. Episodes are limited to 200 steps. [1]

V. Proposed Enhanced Methodology

Fig. 1. Enhanced autonomous-navigation experimental pipeline.

The enhanced methodology follows a five-stage experimental pipeline. First, the grid world and random seeds are initialized to provide a controlled and reproducible environment. Second, the agent explores the environment using ε-greedy action selection, balancing exploration and exploitation. Third, learning is performed using the Q-learning Bellman update [4] or the DQN neural Q-function [5], depending on the selected agent. Fourth, the learned policy is evaluated using success rate, path length, and convergence behavior. Finally, the implementation is extended beyond the fixed 10×10 benchmark through hyperparameter sensitivity, larger-grid scalability, and dynamic-obstacle experiments. This pipeline provides a consistent flow from reproducible initialization to quantitative evaluation and systematic analysis of the enhanced navigation setting.

A. Q-Learning

The tabular agent stores a Q-value for every state-action pair. States are indexed using index(r,c)=(r−1)N+(c−1). The update used by the implementation is the standard temporal-difference rule: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. The documented baseline uses α=0.1, γ=0.9, ε=0.1, 1000 episodes per run, and 20 independent runs.

B. Deep Q-Network

The DQN receives a normalized two-dimensional position representation and uses a fully connected 2→64→64→4 neural network with ReLU hidden activations, following the value-function approximation approach established by DQN [5]. The documented configuration uses learning rate 0.001, γ=0.9, ε=0.2, replay buffer capacity 50,000, batch size 64, and target-network updates every 50 episodes. The DQN experiment uses 500 episodes per run and five runs.

C. Convergence Criterion

Convergence is defined as the first episode at which a 50-episode moving-average return exceeds a threshold of 50. This criterion is used consistently for the reported convergence analysis.

D. Enhanced Experiments

The enhanced implementation adds four analytical dimensions: α sensitivity, ε sensitivity, grid-size scalability, and dynamic-obstacle navigation. In the dynamic environment, the state is expanded to include agent position, obstacle position, and obstacle direction, represented conceptually as (r_a,c_a,r_o,c_o,d), so that the changing obstacle remains part of the Markov state [6]. The overall workflow is summarized in Fig. 1.

VI. Experimental Setup

Parameter	Documented setting
Grid	10×10 benchmark
Start / Goal	(1,1) / (10,10)
Actions	Right, Left, Down, Up
Rewards	Goal +100; obstacle −100; movement −1
Episode limit	200 steps
Q-learning α / γ / ε	0.1 / 0.9 / 0.1
Q-learning training	1000 episodes × 20 runs
DQN architecture	2→64→64→4, ReLU hidden layers
DQN learning rate / γ / ε	0.001 / 0.9 / 0.2
Replay buffer / batch	50,000 / 64
Target update	Every 50 episodes
DQN training	500 episodes × 5 runs
Convergence	50-episode moving average > 50
Software	Python, NumPy 1.26.4, PyTorch 2.8.0
Random seed	42
Device	CPU

VII. Results and Analysis

A. Main Q-Learning and DQN Comparison

Metric	Q-learning	DQN
Mean success rate	98.99 ± 0.14%	49.72 ± 17.61%
Mean successful path length	26.41 ± 20.92 steps	72.79 ± 49.66 steps
Mean convergence	152.55 ± 13.65 episodes	396.67 ± 49.08 episodes*
Greedy demonstration	18 steps	18 steps

*DQN convergence mean is calculated only from the three DQN runs that satisfied the convergence criterion; two of the five runs did not converge within the training horizon.

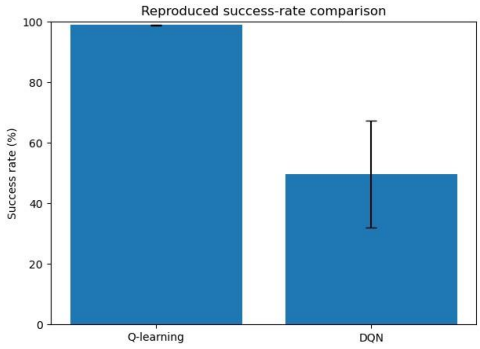

Fig. 2. Reproduced success-rate comparison.

The fixed-grid benchmark strongly favors tabular Q-learning under the tested configuration [1]. Its success rate is near 99% with low between-run variability. The DQN result is considerably lower and more variable. Importantly, the DQN implementation still produced an 18-step greedy demonstration, showing that a successful policy can emerge even though the aggregate training statistics are substantially weaker.

B. Learning-Curve Behavior

The documented learning curves show strongly negative returns during early training followed by improvement and eventual stabilization. Q-learning crosses the convergence threshold substantially earlier than DQN. The DQN curve exhibits greater variability and slower progress, consistent with the lower success rate and larger path-length variance observed across runs.

The following figures are the notebook-generated verification learning curves, showing episode return, the 50-episode moving average, and the convergence threshold of 50.

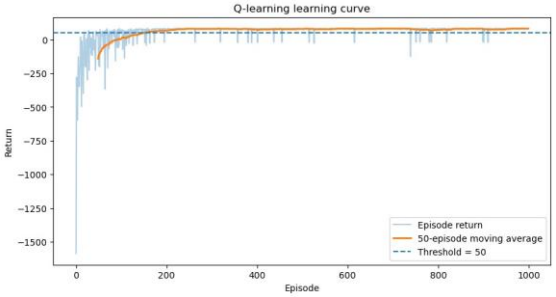

Fig. 3. Q-learning learning curve from the enhanced Python implementation.

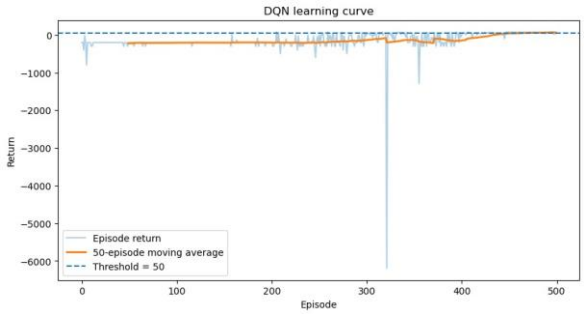

Fig. 4. DQN learning curve from the enhanced Python implementation.

VIII. Greedy Policy Visualization

The final greedy Q-learning policy reaches the goal in 18 moves. The final greedy DQN policy also reaches the goal in 18 moves. Because the Manhattan distance from (1,1) to (10,10) is 18, these individual demonstrations attain the shortest possible path length. They should not be interpreted as the average path length over training runs.

The notebook also renders the final greedy trajectories for both learned policies. These are single-policy demonstrations rather than aggregate training statistics.

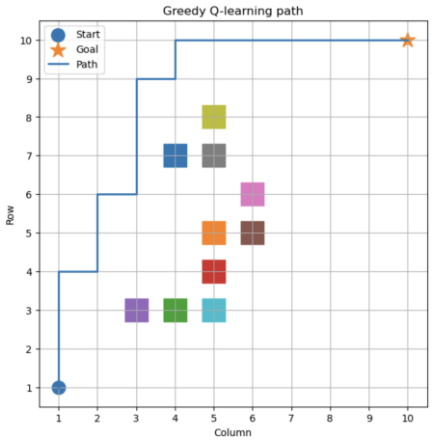

Fig. 5. Greedy Q-learning path; the reported path length is 18 moves.

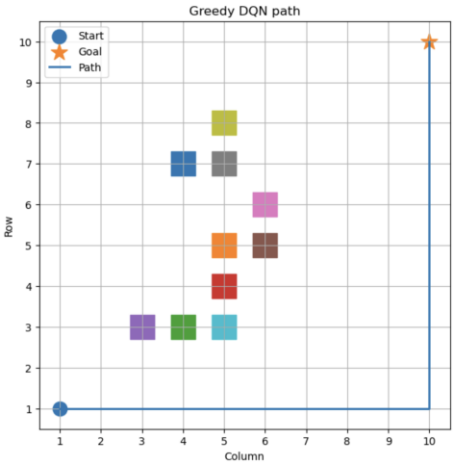

Fig. 6. Greedy DQN path; the reported path length is 18 moves.

IX. Hyperparameter Sensitivity

A. Learning Rate α

α	Success (%)	SD	Successful steps	Convergence
0.01	85.80	1.00	94.13	NaN
0.05	96.67	0.31	44.26	254.67
0.10	98.07	0.31	32.59	136.67
0.20	98.87	0.23	27.12	96.00
0.50	99.47	0.23	24.40	70.00

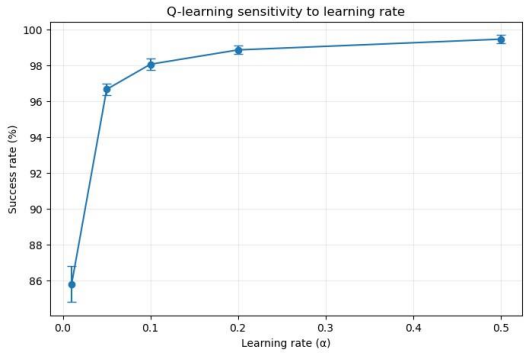

Fig. 7. Q-learning sensitivity to learning rate.

Increasing α from 0.01 to 0.50 produced a strong improvement in the tested configuration. Success increased from 85.80% to 99.47%, successful path length decreased from 94.13 to 24.40 steps, and the reported convergence episode decreased from unavailable at α=0.01 to 70 episodes at α=0.50. The α=0.50 result is 0.48 percentage points above the reproduced baseline success rate of 98.99%; no statistical significance test was performed, so this should be interpreted as an observed experimental improvement rather than proof of superiority.

B. Exploration Rate ε

ε	Success (%)	SD	Successful steps	Convergence
0.05	97.80	0.00	31.38	126.33
0.10	98.07	0.31	32.59	136.67
0.20	97.73	0.50	34.68	229.67
0.30	97.93	0.42	37.96	257.67
0.50	97.53	0.12	47.52	NaN

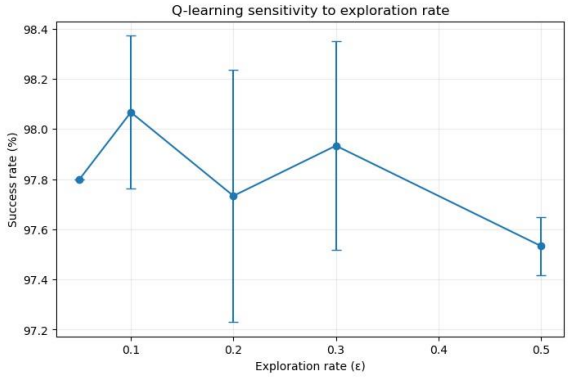

Fig. 8. Q-learning sensitivity to exploration rate.

Success remained close to 98% over the tested ε values, but path length and convergence behavior were more sensitive. Larger exploration rates generally increased successful path length and delayed convergence, while ε=0.05 produced the shortest reported convergence time among the tested values.

X. Grid-Scalability Analysis

Solvable layouts were generated at approximately 10% obstacle density. The scalability experiment used grid sizes 10×10, 15×15, and 20×20, with 500 episodes and three runs per size. The notebook validates that each generated layout has a path from the start to the goal before training.

Grid	States	Obstacles	Success (%)	Successful steps	Convergence
10×10	100	10	98.13 ± 0.46	30.92	148.33
15×15	225	22	96.27 ± 0.12	85.87	416.00
20×20	400	40	96.13 ± 0.50	182.07	NaN

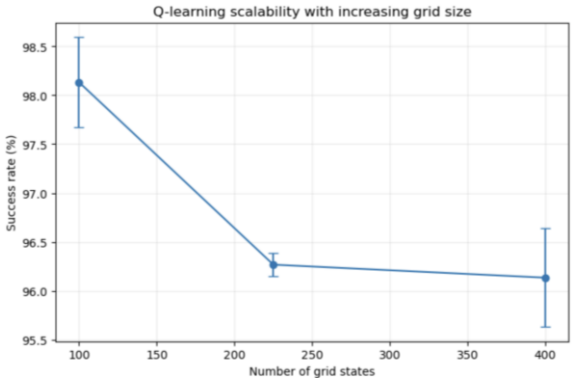

Fig. 9. Q-learning scalability with increasing grid size.

The mean success rate decreases only modestly as the grid grows, from 98.13% at 10×10 to 96.13% at 20×20. However, successful paths become much longer: 30.92, 85.87, and 182.07 steps respectively. Convergence also becomes substantially harder, with no convergence observed in any of the three 20×20 runs within the specified horizon. This indicates that success rate alone does not fully capture the computational and planning difficulty of larger state spaces.

XI. Dynamic-Obstacle Analysis

A dynamic-obstacle experiment was performed using three runs of 500 episodes. The moving obstacle requires the agent to account for both its own position and the obstacle's evolving state. The implemented state therefore includes agent position, obstacle position, and obstacle direction.

Run	Success (%)	Successful steps	Convergence
Run 1	62.2	88.73	NaN
Run 2	62.8	91.61	NaN
Run 3	61.6	88.12	NaN
Mean	62.20	89.49	NaN

The mean success rate falls to 62.20%, considerably below the fixed-obstacle benchmark. None of the three runs satisfies the convergence criterion within 500 episodes. This result demonstrates the difficulty introduced by environmental non-stationarity and the need for richer state representations and/or more advanced learning methods.

XII. Comparative Discussion

The experiments reveal three distinct findings. First, for a small fixed environment, tabular Q-learning is highly effective and converges quickly. Second, the neural-network DQN configuration used here is less reliable under the same benchmark, despite being capable of producing a shortest-path greedy demonstration after training. Third, the principal challenge for Q-learning is not only whether it can reach the goal, but how training difficulty and path inefficiency scale with the number of states and with changing environmental dynamics.

The reproduced DQN success rate of 49.72% differs from the reference paper's reported value of 52.52% [1], and the reproduced convergence behavior also differs from the reference report. The implementation therefore should be viewed as an independent Python reproduction under the documented assumptions, rather than an exact reimplementation of every original experimental detail.

XIII. Limitations

- The benchmark is discrete and small compared with real autonomous-driving environments.
- The main Q-learning and DQN configurations are not exhaustively hyperparameter-tuned.
- The DQN experiment uses only five runs, while Q-learning uses twenty, so uncertainty estimates are not perfectly balanced.
- The dynamic-obstacle experiment is limited to a simplified moving-obstacle scenario.
- No formal statistical significance testing is reported for the hyperparameter comparisons.
- The 18-step greedy path is a single demonstration and is not a substitute for aggregate path statistics.
- Real-world vehicle dynamics, sensor noise, localization error, perception uncertainty, and continuous control are not modeled.

XIV. Future Work

- Extend the environment to continuous state and action spaces and incorporate vehicle dynamics.
- Use prioritized or improved replay strategies and systematic DQN hyperparameter optimization.
- Compare Double DQN, Dueling DQN, and other value-based variants.
- Investigate curriculum learning and transfer learning for larger maps.
- Introduce multiple dynamic obstacles and stochastic obstacle motion.
- Evaluate robustness under sensor noise and partial observability.
- Compare learned policies with A*, Dijkstra, and other classical planners using matched metrics.
- Perform statistical significance testing and confidence-interval reporting over larger independent samples.
- Deploy the navigation policy in a simulated or physical vehicle platform.

XV. Conclusion

This paper documented an enhanced reinforcement-learning study of grid-based autonomous navigation. The reproduced fixed-obstacle experiments show that tabular Q-learning achieves very high success with relatively short paths and substantially earlier convergence than the tested DQN configuration. The additional experiments show that learning-rate selection can materially improve Q-learning behavior, while increasing grid size and introducing dynamic obstacles expose limitations that are not visible in the small fixed benchmark. The dynamic-obstacle mean success of 62.20% and absence of convergence within the training horizon provide clear evidence that changing environments require more sophisticated state representations and learning strategies. Overall, the enhanced implementation provides a useful experimental foundation for moving from simple grid navigation toward more scalable and realistic autonomous-navigation systems.

References

[1] Z. H. A. Aldahlaki, H. F. Hadi Kaze, S. S. Jabbar, and R. S. Hassan, “Comparison of Q-Learning and Deep Q-Network for Autonomous Navigation in Grid-Based Environments,” Journal of Theory, Mathematics and Physics, vol. 5, no. 4, 2026.

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[4] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[5] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[6] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.